Load-Aware UPF Scaling and Performance Evaluation in a 5G Core

Suthar Jainish Chandraprakash Gomtesh
Indian Institute of Technology Gandhinagar
5G and Beyond Projects 2026
suthar.jainish@iitgn.ac.in, gomtesh@iitgn.ac.in

***Abstract*—A single User Plane Function (UPF) in a 5G core carries all user traffic, so its CPU is a bottleneck. Open5GS, the open-source core we use, lets the Session Management Function (SMF) work with several UPFs, but nothing in it decides when to add one. We measure when one UPF degrades and build a small controller that starts a second UPF when the first nears saturation. On a Docker test bed with each UPF capped at 0.1 CPU core, one UPF saturates at about 68 Mbit/s and degrades from about 60 Mbit/s. A controller acting at 80% of the UPF's CPU cap reduces packet loss at 96 Mbit/s offered load from 27.9% (default, one UPF) to 3.9%, without restarting the core; two UPFs running from the start reach 0.0%. Packet traces explain the remaining gap: Open5GS assigns new sessions to UPFs in strict round-robin order and never moves a session. We also show how to remove the second UPF without losing traffic, and which limits of Open5GS remain.**

***Index Terms*—5G core, UPF, Open5GS, UERANSIM, autoscaling, PFCP, GTP-U**

I. Introduction

The UPF forwards all user traffic in a 5G network: uplink packets arrive from the base station over N3 inside GTP-U tunnels and leave towards the data network over N6, while the SMF controls the UPF over N4 using PFCP [1], [2]. When a UPF runs out of CPU, packets are delayed and dropped. The standard allows an SMF to control several UPFs, but it does not say when a second UPF should be added, how new sessions should be spread over the UPFs, or how a UPF can be removed again.

In Open5GS as shipped, UPFs are listed statically in the SMF and nothing starts or stops one in response to load. We call this the *default*. This project asks three questions about it.

- **RQ1.** At what load does one UPF degrade, and which signal shows it early enough to act on?
- **RQ2.** Can a second UPF join a running core, and how does the SMF distribute new sessions over the UPFs?
- **RQ3.** How much does on-demand scaling improve on the default, how does it compare with running two UPFs from the start, and how can the second UPF be removed safely?

Contributions. (i) A measured saturation point and a CPU-based scaling trigger for one UPF. (ii) A lightweight controller that adds and removes a second UPF without restarting the core. (iii) A packet-level verification of how Open5GS distributes sessions, and a comparison of the default, the controller and an always-on pair. (iv) The configuration needed for two UPFs and the practical limits we found.

II. Experimental Setup

A. Test bed

Fig. 1 shows the test bed, run with Docker Compose. The core is Open5GS 2.8.0 [3]; the gNB and simulated UEs are UERANSIM 3.2.6 [4]; the data network (DN) runs one iperf3 server per UE. Each UPF is a separate container with its own N3, N4 and N6 address.

B. CPU cap

UERANSIM cannot generate enough traffic to saturate an uncapped UPF, so we limit each UPF to 0.1 core (1 ms of CPU per 10 ms period). The cap is artificial: absolute Mbit/s values are not real UPF capacities, and the shape of the results is what matters.

C. Workloads and set-ups

Two workloads are used. The *sweep* holds 4 UEs fixed and raises the rate each sends, step by step, to find where one UPF degrades. The *ramp* raises the number of UEs: a new UE joins every 20 s and sends 12 Mbit/s of UDP uplink, up to 8 UEs (96 Mbit/s), which is about 1.4 times the capacity of one UPF but within the capacity of two. The ramp is run in three set-ups: the **default** (one UPF, no scaling), **ours** (one UPF plus a controller that adds the second) and an **upper bound** (both UPFs running before any UE arrives). Each ramp run starts from a fresh stack, and the three set-ups are interleaved.

D. Metrics and measurement

We record delivered throughput and packet loss (both from the iperf3 receivers), RTT (ping through one tunnel), UPF CPU as a share of its cap, the share of 10 ms periods in which the UPF was throttled, and the session count per UPF. CPU and throttling are read through the Docker Engine API and never by running a command inside the UPF: an early version did, and the probe itself added about 4% CPU and 50% throttling at idle. Failed steps are retried and flagged, and the order of loads is shuffled.

E. Host and experiment list

The final results come from one session on a 12-core Linux laptop on AC power with the CPU governor set to `performance` and nothing else running; the load average was 0.3 at the start, peaked at 3.2 during the sweep and stayed below 2.3 during the ramps. Table I lists every experiment and where it is reported.

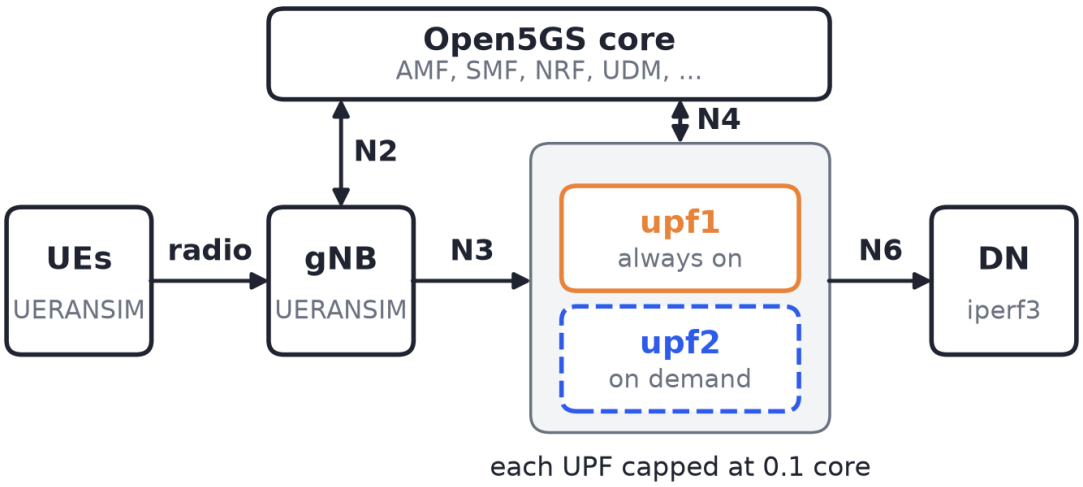


Fig. 1. Test bed. upf1 is always running; upf2 is started on demand. Each UPF is capped at 0.1 CPU core.

TABLE I
EXPERIMENTS. "EARLIER SESSION" IS AN EARLIER RUN ON THE SAME LAPTOP AT A SLOWER CPU SETTING.

ID	Section	What was done	Runs
E1	IV-A	Baseline: one UPF, several UEs, connectivity through the UPF	development runs
E2	IV-B	Load sweep, one UPF, 4 UEs, 10–120 Mbit/s	10 loads × 4 shuffled runs = 40
E3	IV-C	The same sweep in the earlier session	8 loads × 4 = 32
E4	IV-D	Second UPF started at runtime; time until the SMF uses it	the 15 controller runs of E7
E5	IV-E	Session distribution: PFCP on N4 and GTP-U on N3, 6 new UEs	3 runs, 18 sessions
E6	IV-F	Ramp: default vs. controller vs. two UPFs always on	3 set-ups × 3 runs = 9
E7	IV-G	Ramp-down: five ways of ending the ramp	5 scenarios × 3 runs = 15 (the first scenario is E6's "ours")
E8	IV-H	Ramp comparison in the earlier session, 8 UEs at 8 Mbit/s	3 set-ups × 3 runs = 9
E9	IV-H	Scale-down check in the earlier session	1 run, 35 s

III. CONTROLLER DESIGN

Fig. 2 shows the controller, a short Python script that runs once per second. It moves between two states using only UPF CPU and session counts read through the Docker API.

Scale up. If upf1 stays at or above 80% of its CPU cap for 5 s, the controller starts upf2 and waits for the SMF's PFCP association. The threshold is derived from the sweep (E2). **Scale down.** If total UPF CPU stays below 35% of one cap for 20 s *and* upf2 has no sessions, upf2 is stopped. The SMF cannot be told to stop choosing a UPF, so stopping one that still has sessions would cut them. A 30 s cooldown follows every action to prevent flapping. Two options are used in E7: `--empty-down` stops an idle upf2 once upf1 alone is under the 80% limit, and `--drain-timeout` stops upf2 after a fixed time even if it still has sessions.

IV. RESULTS

A. E1: baseline

With one UPF and several UEs, every UE obtained a PDU session, a tunnel interface with an address from the shared UE pool (10.45.0.0/16), and reached the DN through the UPF (ping and iperf3 through the tunnel). This baseline is the starting point of every other experiment.

B. E2: where one UPF degrades (RQ1)

We offered 10–120 Mbit/s from 4 UEs to one UPF, with four shuffled repetitions per load. All 40 runs were valid (no failed iperf3 client, no dead tunnel). Table II and Fig. 3 give the medians.

Finding 1. One UPF saturates at about 68 Mbit/s, and degradation starts at about 60 Mbit/s: throttling jumps from 8% to 67% of periods and the RTT tail from 0.3 to 3.7 ms, and extra load then only adds loss (47% at 120 Mbit/s offered). Above 70 Mbit/s, 67–82% of the lost packets are lost inside

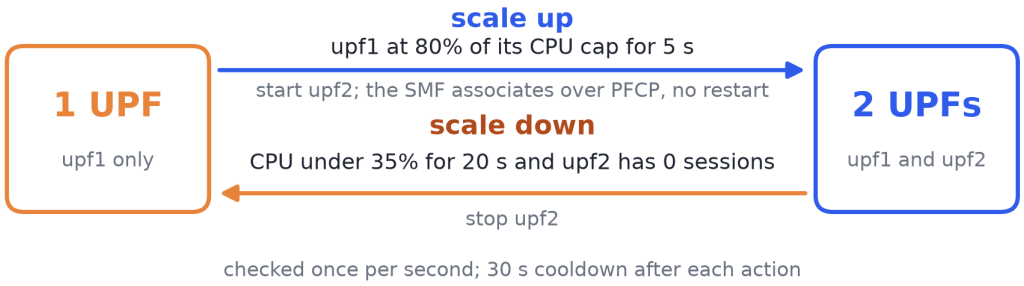


Fig. 2. Controller. It starts upf2 when upf1 stays at 80% of its CPU cap for 5 s, and stops it when total CPU is low and upf2 has no sessions.

TABLE II
E2: ONE UPF AT 0.1 CORE, MEDIANS OF FOUR RUNS PER LOAD.

Offered (Mbit/s)	10	20	30	40	50	60	70	80	100	120
Delivered (Mbit/s)	10.0	20.0	30.0	39.8	49.7	59.4	66.4	67.7	66.5	63.2
Loss (%)	0.0	0.0	0.01	0.4	0.7	1.0	5.1	15.4	33.5	47.3
UPF CPU (% of cap)	23	42	58	71	87	95	98	98	98	98
Throttled periods (%)	0.2	0.9	1.7	1.5	7.6	66.7	99.7	99.7	99.5	99.6
RTT p95 (ms)	0.33	0.30	0.30	0.29	0.29	3.7	10.4	13.3	15.4	16.5

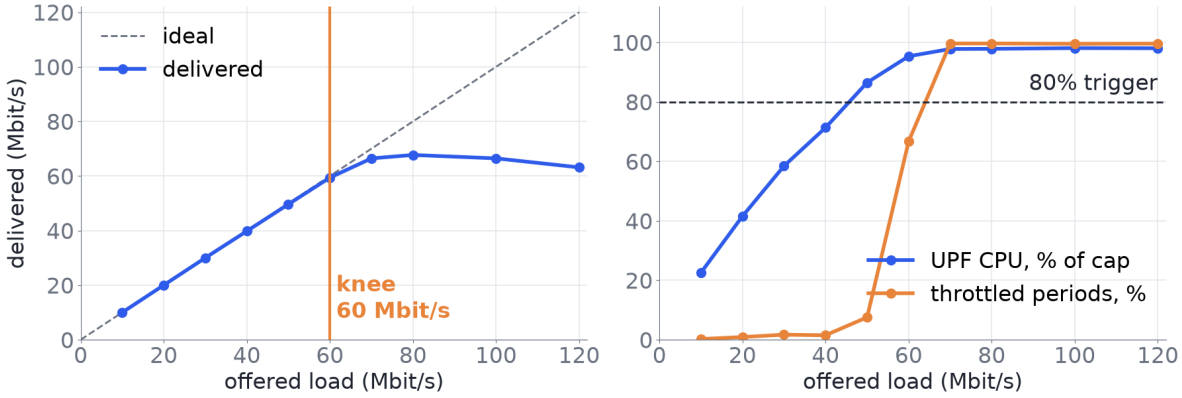


Fig. 3. E2: load sweep for one UPF (medians). Left: delivered traffic follows the offered load up to about 60 Mbit/s, then flattens. Right: UPF CPU and throttled periods; the dashed line is the 80% scale-up trigger.

the UPF (they arrive on N3 but do not leave on N6); the rest is lost elsewhere on the path.

Finding 2. Loss is a weak early warning (1% at 60 Mbit/s), whereas CPU and throttling rise steadily and are usable triggers. A scale-up takes 5 s of holding plus 3–7 s for upf2 to associate, and it helps only sessions created afterwards, so the trigger must fire well before the knee. We use **CPU $\geq$ 80% of the cap for 5 s** (or throttling $\geq$ 50% while CPU $\geq$ 75%), measured with a 3 s moving average. In this sweep that is about 46 Mbit/s, three quarters of the knee and two thirds of the capacity. The threshold is a design choice derived from this sweep; we did not sweep it.

C. E3: the same sweep in the earlier session

Because the cap is CPU time, the Mbit/s that fit in it depend on the host. The earlier session (32 runs, all valid) gave a capacity of 46 Mbit/s instead of 68, a knee at about 40 Mbit/s (62% of periods throttled, loss 1.4%) and 80% of the cap reached at about 30 Mbit/s. The absolute values moved by about 1.5 times, but the 80% trigger again fell at about three quarters of the knee. We therefore express thresholds as a share of the cap and take all reported numbers from a single session.

D. E4: a second UPF joins at runtime (RQ2)

The SMF is started with both UPF addresses in its list while only upf1 runs. It keeps retrying the PFCP association with the absent UPF (about every 11 s) and uses it as soon as it

TABLE III
OPEN5GS SETTINGS NEEDED FOR TWO UPFS.

Parameter	Purpose
`smf.pfcp.client.upf[].address`	Every UPF the SMF may use must be listed (here 10.10.2.20 and .21).
`smf.pfcp.server.address`	Address the UPFs associate and send heartbeats to.
`smf.session[].subnet`	One UE address pool is shared by both UPFs; it cannot be tied to one UPF.
`upf[].dnn / tac / nr_cell_id` (SMF, optional)	Static selectors. Not set here, so the SMF alternates between UPFs.
`upf.pfcp.server.address`	Must equal the address in the SMF list.
`upf.gtpu.server.address`	N3 address sent to the gNB; uplink is tunnelled to it.
`upf.session[].subnet`	Each UPF must route the whole UE pool into its TUN device.
NAT on N6 (`iptables MASQUERADE`)	Both UPFs serve one pool, so the data network cannot tell which UPF owns an address. NAT sends replies back to the sending UPF.
Compose: own N3/N4/N6 IPs	Separate interfaces per UPF; a profile keeps upf2 off until needed.

TABLE IV
E5: UPF CHOSEN BY THE SMF FOR EACH NEW SESSION, IN ORDER (SAME IN ALL 3 RUNS). THE GTP-U TRAFFIC OF EVERY SESSION REACHED THAT UPF, 6 OF 6 PER RUN.

Session	1	2	3	4	5	6
UPF	upf2	upf1	upf2	upf1	upf2	upf1

answers, so no restart is needed. Timed by hand, upf2 started at 14:14:17.9 and was associated at 14:14:21.7, after 3.8 s. In all 15 controller runs of E7, which include the three controller runs of E6, upf2 was ready 3.3–6.5 s after the controller started it. Table III lists the settings required; the AMF and gNB need no UPF-specific settings, because the gNB learns each session's N3 address from the core.

E. E5: how sessions are distributed (RQ2)

We captured PFCP on N4 and GTP-U on N3 (inside the core and gNB containers, so the UPFs pay nothing for the capture) with both UPFs up and 6 new UEs per run, for 3 runs. In all 18 sessions the SMF chose the UPFs strictly in turn (Table IV), and in every case the GTP-U packets carrying the session's tunnel endpoint ID (TEID) reached the UPF named in the PFCP Session Establishment. **Finding 3:** Open5GS places new sessions in round-robin order, independent of load.

F. E6: scaling up, ramp comparison (RQ3)

Table V and Fig. 4 compare the three set-ups; Table VI lists every run. Values are the steady state, from 10 s after the last UE joined until the first leaves.

Finding 4. The controller cuts loss at 96 Mbit/s from 27.9% to 3.9% (86% less) and RTT from 8.7 to 3.7 ms; the ranges of the three set-ups do not overlap. It fired at 69–70 s in all three runs, right after the fourth UE joined (48 Mbit/s, upf1 above 80%). Per-UE loss, for example 1.9% on average for

TABLE V
E6: STEADY STATE AT 96 MBIT/S OFFERED, MEAN (MIN–MAX) OF 3 RUNS.

Set-up	Delivered (Mbit/s)	Loss (%)	RTT (ms)	UEs on upf2
Default (1 UPF)	69.3 (68.8–69.6)	27.9 (27.5–28.3)	8.7 (8.4–9.1)	0 of 8
Ours (controller)	92.3 (91.2–93.6)	3.9 (2.5–5.0)	3.7 (3.5–4.1)	2 of 8
Upper bound (2 UPFs)	96.0 (96.0–96.0)	0.0	0.8 (0.7–1.0)	4 of 8

TABLE VI
E6: EVERY RUN. CPU IS A SHARE OF THE UPF'S CAP.

Run	Deliv. (Mbit/s)	Loss (%)	RTT (ms)	upf1 CPU	upf2 CPU	UEs on upf2	Scale-up at (s)
default 1	68.8	28.3	9.1	100	–	0	–
default 2	69.6	27.5	8.5	100	–	0	–
default 3	69.4	27.7	8.4	100	–	0	–
ours 1	91.2	5.0	4.1	100	24	2	70
ours 2	93.6	2.5	3.5	100	22	2	69
ours 3	92.0	4.2	3.7	100	23	2	70
upper bound 1	96.0	0.0	0.7	92	88	4	–
upper bound 2	96.0	0.0	1.0	88	93	4	–
upper bound 3	96.0	0.0	0.8	90	85	4	–

the UEs on upf1, is taken over each UE's whole run, including the time before saturation, so it is lower than the steady-state figure.

Why not 0.0%? Two properties of Open5GS from Finding 3 explain it, and simple arithmetic confirms them. The four UEs placed before the scale-up stay on upf1, and new sessions keep alternating, so UEs 5 and 7 land on upf2 but UEs 6 and 8 still land on upf1. Upf1 then carries 6 UEs, 72 Mbit/s against a capacity of about 68, which is roughly 4 Mbit/s lost out of 96, i.e. about 4%. The two UEs on upf2 see no loss. With the upper bound, each UPF carries 48 Mbit/s and nothing is lost.

G. E7: scaling down (RQ3)

After 60 s at full load, one UE stops its traffic and deregisters cleanly every 20 s. UEs 5 and 7 are on upf2 in every run. We compare five ways of ending the ramp, three runs each (Table VII, Fig. 5); "fits on one" means an offered load below 45 Mbit/s, where upf1 alone stays under the 80% limit.

Finding 5. Waiting for 0 sessions loses nothing in all 12 runs that waited (mean loss at most 0.06%), but the wait has no upper bound: in the fourth scenario one 12 Mbit/s UE kept upf2 running for 150 s after one UPF was enough, and it would have stayed forever had the UE not left. An empty upf2 is waste, so stopping it early saves 40 s at no cost; the controller still waits while upf1 is busy. Forcing the stop bounds the wait but costs traffic: the SMF noticed the missing UPF only through missed PFCP heartbeats, the affected UE re-established its session on upf1 with a *new IP address*, and it had no traffic for 10.7 s (10–12 s) in 3 of 3 runs. No scale-up fired during any of the 15 ramp-downs, and none while load was high.

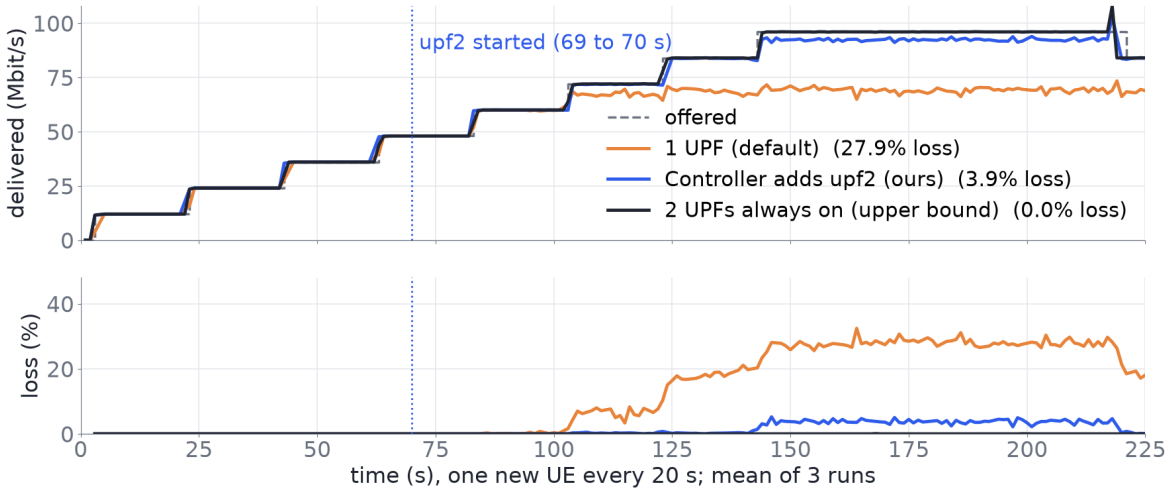


Fig. 4. E6: ramp to 96 Mbit/s offered (mean of 3 runs). With one UPF, delivered traffic stops near 69 Mbit/s and loss climbs to about 28%. The controller starts upf2 at 69–70 s, while the load is still below capacity.

TABLE VII
E7: SCALE-DOWN SCENARIOS (MEANS OF 3 RUNS, IN SECONDS FROM THE START OF THE RAMP). "IDLE" IS HOW LONG UPF2 RAN WITH NO SESSIONS; "EXTRA" IS HOW LONG BOTH UPFS KEPT RUNNING AFTER THE LOAD FITTED ON ONE.

Scenario	upf2 empty	upf2 stopped	Idle	Extra	Cost
Wait for 0 sessions, UEs leave in arrival order	340	365	25	64	none
Wait for 0 sessions, UEs leave in reverse order	280	366	86	65	none
Reverse order, idle upf2 stopped early (`--empty-down`)	280	326	45	25	none
Wait for 0 sessions, upf2's UEs leave last	451	451	0	150	none
Forced stop 30 s after low load (`--drain-timeout`)	–	381	0	80	one UE cut 10.7 s, new IP

H. E8, E9: checks in the earlier session

The ramp comparison was first run in the earlier session with 8 UEs at 8 Mbit/s each (64 Mbit/s offered, 9 runs, none with failed UEs). One UPF gave 19.3% loss and 10.0 ms RTT, the controller 0.5% and 3.0 ms, and two UPFs from the start 0.0% and 0.7 ms; the controller fired at 71–72 s in all three runs. The picture is the same as in E6 at different absolute values. In E9, a 35 s run with both UPFs up, low load and one UE on upf2, the controller waited while upf2 still had that session and stopped upf2 at the first sample after the UE had deregistered.

V. DISCUSSION

Limits of dynamic scaling in Open5GS. (1) Selection is round-robin, not load-aware; the only selectors are static (DNN, TAC, cell ID), so after a scale-up every other new UE still goes to the saturated UPF. (2) Existing sessions never move, so scaling relieves only future load. (3) A UPF cannot be drained: the SMF cannot be told to stop choosing it, and a UE that dies without deregistering left its session on upf2 for over 60 s. Scale-down is therefore all-or-nothing, either wait or cut. (4) Scale-up is slow: 5 s of holding plus 3–7 s of association. (5) Because both UPFs share one UE pool, the data network needs NAT on N6.

Measurement pitfalls. Early runs were discarded for three reasons. A command run inside the UPF to read its CPU distorted the CPU it measured (E2 setup). Hung iperf3 clients and dead UE tunnels contaminated some steps, so iperf3 is now reset at every step and failed steps are retried and flagged. UERANSIM also dropped radio links under stress. Finally, one repetition was discarded after other applications exhausted the host's memory.

Threats to validity. The 0.1-core cap is artificial. Traffic is uplink UDP from simulated UEs, with one gNB, one slice and at most two UPFs. Each set-up has three repetitions, though the ranges are far apart, and all final numbers come from one host in one session; E3 and E8 show the same picture at different absolute values. The 80% threshold was chosen, not tuned.

VI. CONCLUSION

At a 0.1-core cap one UPF carries about 68 Mbit/s and degrades from about 60. A controller that uses only UPF CPU, with a trigger at three quarters of the way to the knee, adds a second UPF without restarting the core and removes 86% of the loss at 96 Mbit/s. The remaining loss comes from Open5GS itself, which places sessions in strict round-robin order and never moves them. Removing the second UPF is safe if the controller waits for it to empty, but the wait is unbounded, and forcing it costs the affected UE about 11 s

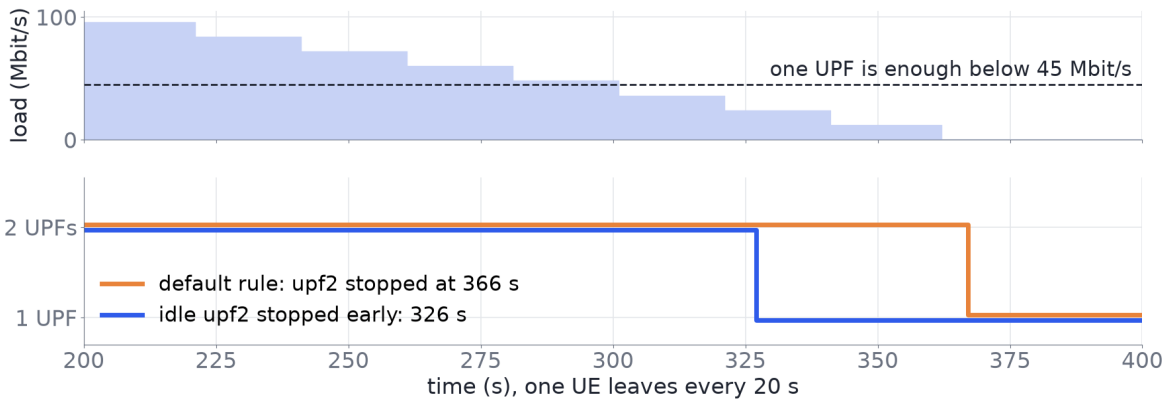


Fig. 5. E7: ramp-down (repetition 2). Top: the load falls as UEs leave. Bottom: number of running UPFs under the default rule and with the early stop of an idle upf2.

and its IP address. Next steps are to re-establish some PDU sessions after a scale-up and to make UPF selection load-aware in the SMF.

Code and data. Everything is in the project repository, submitted separately. Table II comes from `results/final/threshold.csv`; Tables V–VII from the per-second timelines in `results/final/ramp/` (`summary.md` holds every run, `host.txt` the host state); Table IV from the N4 and N3 captures in `results/linux/trace_r1..r3/` (`.pcap`, openable in Wireshark); E3, E8 and E9 from `results/linux/`. The whole final session reruns with `final.sh`.

References

[1] 3GPP, "System architecture for the 5G System (5GS)," TS 23.501.
[2] 3GPP, "Interface between the Control Plane and the User Plane nodes (PFCP)," TS 29.244.
[3] Open5GS, https://open5gs.org.
[4] UERANSIM, https://github.com/aligungr/UERANSIM.